🔷 EXPERIMENT 3 — FULL GIT WORKFLOW

✅ STEP 0: Open Git Bash

cd ~

✅ STEP 1: Create project folder

```
mkdir data-science-project3
cd data-science-project3
```

✅ STEP 2: Initialize Git

```
git init
```

✅ STEP 3: Create branch GIT-EXAM-1

```
git checkout -b GIT-EXAM-1
```

✅ STEP 4: Create program2.py (Palindrome)

```
touch program2.py
notepad program2.py
```

✍️ Code:

```
text = input("Enter string: ")

if text == text[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

✅ STEP 5: Create program1.py (Squares 1–10)

```
notepad program1.py
```

✍️ Code:

```
for i in range(1, 11):
    print(i, "square =", i*i)
```

✅ STEP 6: Commit GIT-EXAM-1 branch

```
git add .
git commit -m "GIT-EXAM-1: palindrome and squares programs"
```

✅ STEP 7: Connect GitHub

```
git remote add origin https://github.com/your-username/repo3.git
```

✅ STEP 8: Push GIT-EXAM-1

```
git push -u origin GIT-EXAM-1
```

✅ STEP 10: Create GIT-EXAM-2 branch

```
git checkout -b GIT-EXAM-2
```

🔷 STEP 11: Pull from GIT-EXAM-1

```
git pull origin GIT-EXAM-1
```

✔️ This brings code from branch 1 into branch 2

✅ STEP 12: program1.py

Open:

```
notepad program1.py
```

Change code (DIFFERENT VERSION):

```
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

⚠️ STEP 13: Create conflict scenario (IMPORTANT)

Switch to GIT-EXAM-1 and also modify same file differently:

```
git checkout GIT-EXAM-1
notepad program1.py
```

Different edit:

```
for i in range(1, 11):
    print("Cube of", i, "is", i*i*i)
```

✔ Now BOTH branches modified SAME file differently → conflict ready

---

🔷 STEP 14: Commit in GIT-EXAM-2

```
git checkout GIT-EXAM-2
git add .
git commit -m "Modified program1 for conflict demo"
git push -u origin GIT-EXAM-2
```

🔷 STEP 15: Commit in GIT-EXAM-1

```
git checkout GIT-EXAM-1
git add .
git commit -m "Different modification for conflict demo"
git push -u origin GIT-EXAM-1
```

🔷 STEP 16: Merge comparison (DIFF)

```
git diff GIT-EXAM-1 GIT-EXAM-2
```

🔷 STEP 17: View history

```
git log --oneline --graph --all
```

🔷 STEP 18: Create Pull Request (IMPORTANT)

On GitHub:

Step:

1. Open repo
2. Click Pull Request
3. Set:
    - Base: GIT-EXAM-1
    - Compare: GIT-EXAM-2
4. Click Create PR
5. Click Merge Pull Request

🔷 FINAL STRUCTURE

🟢 GIT-EXAM-1

program1.py → squares
program2.py → palindrome

🔵 GIT-EXAM-2

program1.py → modified version (conflict demo)
program2.py → palindrome